



Medo-Veda

One-page app summary based on repo evidence

WHAT IT IS

Medo-Veda is an AI-powered health app for analyzing product ingredients and clinical-style scan inputs. The repo shows a React frontend paired with an Express backend that runs a multi-agent analysis pipeline and returns verdicts, scores, and reports.

WHO IT'S FOR

People who want ingredient safety insights tailored to a personal health profile; the product flows and dashboard also suggest a user persona focused on tracking scan history and health-oriented recommendations.

HOW TO RUN

- Backend: ``cd backend` then `npm install` and `npm start``.
- Frontend: ``cd frontend` then `npm install` and `npm run dev``.
- Frontend API target: set ``VITE_API_URL`` if the backend is not at ``http://127.0.0.1:3001``.
- Required backend env values are implied by code and README: Gemini/AI, PostgreSQL, Redis, and Cloudinary credentials.
- Database migration/seed order: Not found in repo.

WHAT IT DOES

- Scans product data from image upload, manual text entry, or voice input.
- Uploads image files, then runs asynchronous analysis with polling status updates.
- Builds a stored user persona with age, gender, conditions, goals, diet, weight, and height.
- Generates report verdicts, health scores, recommendations, evidence, and alternatives.
- Shows dashboard summaries plus scan history and report views in the frontend.
- Caches prior analyses in Redis and stores long-term records in PostgreSQL.

HOW IT WORKS

- Frontend: React + Vite single-page app with routes for landing, auth, dashboard, scan, history, and report pages.
- API: Express server exposes auth, profile, dashboard, scan, image extraction, and async status/result endpoints.
- Upload flow: Multer writes temp files; image scans upload to Cloudinary before analysis.
- Analysis flow: scan controller checks Redis cache, loads the user's persona from PostgreSQL, then calls the orchestrator.
- Pipeline: the orchestrator runs vision extraction for images plus 9 agent modules for persona, product, ingredients, claims, recommendations, evidence, alternatives, verdict, and presentation.
- Persistence: PostgreSQL stores personas and scans; Upstash Redis stores hot cache entries and scan lookups.

Repo gaps called out explicitly

Migration and seeding sequence are not documented as a single startup workflow in the repo.